Workspace Project: Architectural & Design Decisions

1. System Bootstrap & Global Admin

- **Automatic Seeding:** On server startup, the system verifies the existence of the `admin@workspace.com` user and auto-provisions this "Global Admin" account if it is missing.
- **Credentials:** Default email is `admin@workspace.com` with a user-configurable default password.
- **Restricted Capabilities:** To prevent catastrophic data loss, only the Global Admin possesses the system authority to create and delete entire projects.

2. Technology Stack & Data Management

- **Backend:** Built with Node.js and TypeScript to ensure robust type safety across all controllers.
- **Database:** MySQL serves as the relational backbone, utilizing strict foreign key constraints with `ON DELETE CASCADE` to guarantee no orphaned tasks or comments are left behind.
- **Payload Handling:** Backend request limits for JSON and URL-encoded data were explicitly increased to 1MB to accommodate direct Base64 string encoding for user profile photos.

3. Workflow & Work-in-Progress (WIP) Limits

- **Dynamic Status Columns:** Status stages are managed dynamically via a dedicated `columns` table rather than hardcoded enums.
- **Default WIP Limits:** New boards default to a capacity of 3 for 'In Progress' and 1 for 'In Review'. Any custom column created manually defaults to a WIP limit of 3.
- **Capacity Enforcement:** The system strictly validates the task count of the target column before permitting a 'Move' or 'Create' operation, preventing workflow bottlenecks.

4. Transitions & Custom Workflows

- **Default Logic:** Adjacency movement (moving a task a distance of 1 column) is permitted by default.
- **Custom Rules:** Non-adjacent forward movements, or any backward movements, require explicit configuration in the `allowed_workflows` table.
- **Validation:** Every task-move operation queries these predefined rules to strictly enforce board governance and process flow.

5. Story & Sub-task Architecture

- **Hierarchy:** A 'Story' functions as a parent task, encapsulating 'Task' and 'Bug' types as children.

- **Status Derivation:** A Story's status is dynamically derived using "Minimum Task Precedence"—meaning the Story sits in the column of its furthest-behind child task.
- **Cleanup:** If all child tasks are deleted, the parent Story reverts to an 'Unplaced' status.

6. Role-Based Access Control (RBAC)

- **Project Roles:** Access is divided into Project Admin, Project Member, and Project Viewer.
- **Assignment Restrictions:** Project Members are restricted from assigning tasks or modifying existing assignments; team allocation is strictly reserved for Admins.
- **Proactive UI:** The frontend transparently disables assignee selection for Project Members, utilizing tooltip feedback to prevent user frustration and unnecessary API errors.

7. Immutable Audit Logs

- **Point-in-Time Snapshots:** Instead of relying on live table joins, the audit log captures static snapshots of key states (e.g., `is_closed`, `is_resolved`) at the exact moment an event occurs.
- **Anti-Cascading:** This guarantees historical accuracy, preventing a current action (like closing a task) from retroactively altering the historical context of past logs.

8. Profile & Avatar Storage

- **Direct Storage Model:** Optimized for a low expected user volume (<5), user avatars are stored directly as Base64 strings within a `LONGTEXT` database column.
- **Rationale:** This architectural choice removes the overhead of managing external file servers or S3 buckets, keeping the infrastructure completely self-contained.

9. Security & Authentication

- **Dual JWT Strategy:** Employs short-lived (15-minute) Access Tokens for active security and long-lived (7-day) Refresh Tokens stored in a dedicated table for persistent sessions.
- **XSS Mitigation:** Both tokens are secured using `httpOnly: true` cookies, effectively neutralizing Cross-Site Scripting (XSS) extraction risks.
- **Cryptography:** Passwords are mathematically secured using `bcrypt` with a salt round of 10.

10. Silent Authentication (Refresh Interceptors)

- **Interceptor Pattern:** A frontend API request wrapper automatically catches `401 Unauthorized` responses.
- **Silent Renewal:** Before throwing an error, the wrapper seamlessly triggers an `/auth/refresh` request to obtain a new access token, then transparently retries the failed request to ensure a frictionless user experience.

11. Soft and Hard Deletion Synergy

- **Archival (Soft Delete):** Projects utilize a lightweight boolean toggle (`is_archived`) to hide inactive workspaces while fully preserving audit trails and historical references.
- **Permanent Destruction:** Explicit "Delete Project" actions execute a hard database `DELETE`. Leveraging MySQL's native `ON DELETE CASCADE`, this cleanly purges all associated tasks, comments, and relationships in a single transaction.

12. Real-Time Synchronization via Polling

- **Simplified Architecture:** To minimize backend complexity, the system uses lightweight frontend short polling (e.g., 30-second intervals) instead of WebSockets or Server-Sent Events (SSE).
- **Reliability:** This guarantees consistent UI updates while eliminating the connection-drop overhead and state-management complications associated with persistent sockets.

13. Graceful UI Degradation & Styling

- **Aesthetic Direction:** The UI embraces standard, vanilla CSS techniques (Flexbox, predefined hex variables) and deliberately eschews heavily abstracted utility-class frameworks.
- **Modular Styling:** Global layouts are defined centrally (`DashBoard.css`), while component-specific styles use CSS Modules (`*.module.css`). This guarantees zero class-name collisions without requiring heavy compiler plugins.

14. Stable Sequential API Execution

- **Predictable Loading:** Where multiple fetch calls are required, the frontend deliberately favors sequential `await` operations over concurrent `Promise.all` patterns.
- **Rationale:** This acts as a throttle to prevent database connection bottlenecking and establishes a highly predictable loading waterfall mechanism.

15. Story Status Retention

- **Historical Persistence:** A Story's final status is permanently retained for tracking purposes, maintaining its overarching state even if its final concluding child task is subsequently moved or modified.

16. Strict Story Creation Bounds

- **Dedicated Creation:** Stories can only be generated via the dedicated Stories interface.
- **Immutability:** Standard tasks or bugs cannot be retroactively converted into Stories, maintaining strict structural boundaries within the task hierarchy.